

MANUAL DE EJECUCIÓN EN VM LINUX

Despliegue con Docker

Autor: Oscar A Perez Perez

ÍNDICE DE CONTENIDO

1. Objetivo
2. Resumen del Proyecto
3. Requisitos Básicos en la VM Linux
4. Instalación de Docker
5. Preparación del Proyecto
6. Ejecución Básica con Docker Compose
7. Servicios del Sistema
8. Primer Arranque y Datos
9. Comandos Útiles
10. Actualización del Código (Rebuild)
11. Actualizar Imágenes en Producción
12. Consideraciones Avanzadas
13. Solución de Problemas
14. Estructura de Archivos
15. Comandos Resumen (Cheat Sheet)
16. Preguntas Frecuentes
17. Precauciones Críticas

1. OBJETIVO

Este manual explica paso a paso cómo ejecutar PlaneacionPUA en una VM Linux usando Docker y los archivos ya configurados del proyecto. Incluye requisitos, instalación, puesta en marcha, verificaciones, y qué hacer después de modificar el código y actualizar las imágenes.

2. RESUMEN DEL PROYECTO

Componente	Descripción
Backend	Laravel (PHP-FPM) en contenedor
Frontend	React + Vite compilado y servido por Nginx
Base de Datos	MySQL 8.0
Cache/Colas	Redis
Orquestación	docker-compose.yml en la raíz del proyecto

3. REQUISITOS BÁSICOS EN LA VM LINUX

3.1 Hardware Recomendado

Recurso	Especificación
CPU	2 núcleos mínimo
RAM	4 GB mínimo (8 GB recomendado)
Disco	20 GB mínimo

3.2 Software Requerido

- Docker Engine
- Docker Compose (plugin)
- Git (opcional, si se clonará el repositorio en la VM)

3.3 Sistemas Probados

Este manual ha sido probado en las siguientes distribuciones de Linux:

- Ubuntu 22.04 / 24.04
- Debian 12
- Rocky Linux / AlmaLinux 9

4. INSTALACIÓN DE DOCKER (EJEMPLO UBUNTU/DEBIAN)

Nota: Ejecutar como usuario con permisos sudo. En otras distribuciones, consultar la documentación oficial de Docker.

Paso 1: Actualizar repositorios

```
sudo apt-get update
```

Paso 2: Instalar dependencias

```
sudo apt-get install -y ca-certificates curl gnupg
```

Paso 3: Agregar llaves y repositorio de Docker

```
sudo install -m 0755 -d /etc/apt/keyrings
```

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Paso 4: Instalar Docker Engine y Compose plugin

```
sudo apt-get update
```

```
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Paso 5: Validar instalación

```
docker --version
```

```
docker compose version
```

Paso 6: Permitir ejecutar Docker sin sudo (Opcional)

```
sudo usermod -aG docker $USER
```

Importante: Cerrar sesión y volver a entrar para que los cambios surtan efecto.

5. PREPARACIÓN DEL PROYECTO EN LA VM

5.1 Copiar o Clonar el Proyecto

```
git clone <URL_DEL_REPOSITORIO>
```

```
cd PlaneacionPUA
```

5.2 Verificar Archivo docker-compose.yml

Asegurarse de que existe el archivo `docker-compose.yml` en la raíz del proyecto.

5.3 Crear Archivo .env para Docker

En la raíz del proyecto, crear un archivo `.env` con las variables que usa `docker-compose.yml`.

Ejemplo de configuración:

```
APP_URL=http://<IP_DE_LA_VM>
```

```
CORS_ALLOWED_ORIGINS=http://<IP_DE_LA_VM>
```

```
DB_DATABASE=pua_db
```

```
DB_USERNAME=user
```

```
DB_PASSWORD=password
```

Nota: Si no se crea el archivo `.env`, se usarán los valores por defecto del `docker-compose.yml`.

5.4 Verificar Carpeta de Base de Datos

Verificar que la carpeta `database/` contenga el archivo SQL de inicialización. El contenedor MySQL ejecuta automáticamente scripts en `./database` al primer arranque.

6. EJECUCIÓN BÁSICA CON DOCKER COMPOSE

Importante: Todos los comandos se ejecutan desde la raíz del proyecto.

6.1 Construir Imágenes

```
docker compose build
```

6.2 Levantar Contenedores

```
docker compose up -d
```

6.3 Verificar Estado de Servicios

```
docker compose ps
```

6.4 Ver Logs

```
docker compose logs -f
```

6.5 Acceso a la Aplicación

Servicio	URL de Acceso
Frontend (Nginx)	http://<IP_DE_LA_VM>
API (Laravel)	http://<IP_DE_LA_VM>/api

Nota: El frontend sirve el contenido estático y reenvía /api a PHP-FPM en el servicio backend.

7. SERVICIOS DEL SISTEMA

7.1 Frontend

- Construye React con Vite
- Sirve /usr/share/nginx/html con Nginx en el puerto 80
- Redirige /api al backend vía FastCGI

7.2 Backend

- PHP 8.2 FPM
- Ejecuta migraciones automáticamente al iniciar (docker-entrpoint.sh)
- Limpia cache de Laravel al inicio
- Se conecta a MySQL y Redis

7.3 Base de Datos (db)

- MySQL 8.0
- Persiste datos en volumen db_data
- Ejecuta scripts SQL en ./database la primera vez

7.4 Redis

Redis para cache y colas

8. PRIMER ARRANQUE Y DATOS

- El contenedor backend ejecuta `php artisan migrate --force` al iniciar
- Si hay scripts SQL en ./database, MySQL los ejecuta la primera vez
- Si necesita datos de prueba, cree un seeder y ejecútelo dentro del contenedor backend:

```
docker compose exec backend php artisan db:seed --force
```


9. COMANDOS ÚTILES

Acción	Comando
Entrar al contenedor backend	<code>docker compose exec backend bash</code>
Ejecutar comandos Artisan	<code>docker compose exec backend php artisan <comando></code>
Ver estado de la base de datos	<code>docker compose exec db mysql -u<user> -p<pass> <db></code>
Detener servicios	<code>docker compose down</code>
Detener y borrar volúmenes (BORRA DB)	<code>docker compose down -v</code>

10. DESPUÉS DE MODIFICAR EL CÓDIGO (REBUILD)

Cuando cambie el código fuente, es necesario reconstruir las imágenes. Proceso recomendado:

10.1 Procedimiento General

1. Detener servicios: `docker compose down`
2. Reconstruir imágenes: `docker compose build`
3. Levantar nuevamente: `docker compose up -d`

Rebuild sin caché: Para cambios grandes, use `docker compose build --no-cache`

10.2 Notas por Tipo de Cambio

Tipo de Cambio	Acción Requerida
Cambios en Frontend	Rebuild del servicio frontend (genera nuevo build estático)
Cambios en Backend	Rebuild del servicio backend. Migraciones se ejecutan automáticamente
Cambios en docker-compose.yml o Dockerfile	Siempre reconstruir imágenes
Cambios en .env o nginx.conf	Reconstruir imagen correspondiente y reiniciar contenedores

11. ACTUALIZAR IMÁGENES EN PRODUCCIÓN

Flujo recomendado para actualizar la aplicación en la VM de producción:

4. Obtener cambios del repositorio: `git pull`
5. Reconstruir imágenes: `docker compose build`
6. Reiniciar servicios: `docker compose up -d`
7. Verificar logs y salud: `docker compose logs -f`

12. CONSIDERACIONES AVANZADAS

12.1 Puertos y Firewall

Asegurarse de que el puerto 80 de la VM esté abierto en el firewall.

12.2 Base de Datos Persistente

El volumen `db_data` mantiene los datos aunque se reinicien contenedores.

12.3 Permisos en Storage

El Dockerfile del backend asegura permisos en `/var/www/storage` y `/var/www/bootstrap/cache`.

12.4 Variables de Entorno

Use un archivo `.env` en la raíz para configurar `APP_URL`, `DB_*` y `CORS_ALLOWED_ORIGINS`.

12.5 Migraciones Automáticas

Si se requiere más control, ajuste `docker-entrypoint.sh`.

12.6 Backups

Para respaldar la base de datos, realizar dumps desde el contenedor `db`.

13. SOLUCIÓN DE PROBLEMAS BÁSICOS

Problema	Solución
El frontend no carga	Revisar logs: docker compose logs frontend Verificar puerto 80 y firewall
Errores 500 en API	Revisar logs backend: docker compose logs backend Verificar variables DB y conexión a MySQL
La base de datos no inicia	Revisar logs db: docker compose logs db Verificar espacio en disco y credenciales
Cambios no se reflejan	Rebuild de imágenes y reinicio de contenedores

14. ESTRUCTURA DE ARCHIVOS RELEVANTES

Archivo	Descripción
docker-compose.yml	Define todos los servicios
backend/Dockerfile	Imagen PHP-FPM y dependencias
backend/docker-entrypoint.sh	Migraciones y limpieza de cache
frontend/Dockerfile	Build React y Nginx
frontend/nginx.conf	Sirve SPA y reenvía /api

15. COMANDOS RESUMEN (CHEAT SHEET)

Operación	Comando
Construir	docker compose build
Arrancar	docker compose up -d
Ver logs	docker compose logs -f
Detener	docker compose down
Rebuild completo	docker compose down docker compose build docker compose up -d

16. PREGUNTAS FRECUENTES

16.1 ¿Qué pasa con los datos de la base de datos cuando reinicio los contenedores?

Los datos de la base de datos se **mantienen seguros** porque están almacenados en un volumen Docker persistente llamado `db_data`. Cuando ejecutas `docker compose down` y luego `docker compose up -d`, los datos permanecen intactos. El volumen solo se elimina si usas explícitamente el flag `-v` (`docker compose down -v`).

16.2 ¿Cómo hago un respaldo de la base de datos?

Para crear un respaldo completo de la base de datos, ejecuta:

```
docker compose exec db mysqldump -u<usuario> -p<password>  
<basedatos> > backup_$(date +%Y%m%d).sql
```

16.3 ¿Cómo restauro un respaldo de la base de datos?

Para restaurar desde un archivo de respaldo:

```
docker compose exec -T db mysql -u<usuario> -p<password>  
<basedatos> < backup_20250101.sql
```

16.4 ¿Necesito reconstruir las imágenes cada vez que cambio el código?

Sí, siempre que modifiques el código fuente del frontend o backend, necesitas reconstruir las imágenes con `docker compose build`. Esto es necesario porque Docker crea imágenes inmutables que incluyen el código en el momento de la construcción.

16.5 ¿Puedo ver los logs de un servicio específico?

Sí, puedes ver los logs de servicios individuales:

```
docker compose logs -f frontend
```

```
docker compose logs -f backend
```

```
docker compose logs -f db
```

16.6 ¿Cómo accedo a la base de datos directamente?

Para conectarte al cliente MySQL dentro del contenedor:

```
docker compose exec db mysql -u<usuario> -p<password>  
<basedatos>
```

16.7 ¿Qué hago si el puerto 80 ya está en uso?

Si el puerto 80 está ocupado, puedes cambiar el puerto de mapeo en el archivo `docker-compose.yml`. Busca la sección del servicio frontend y modifica el mapeo de puertos de `80:80` a `8080:80` (o cualquier otro puerto disponible). Luego accede a la aplicación en `http://<IP_DE_LA_VM>:8080`

16.8 ¿Cómo ejecuto comandos de Laravel (Artisan)?

Todos los comandos de Artisan se ejecutan dentro del contenedor backend:

```
docker compose exec backend php artisan <comando>
```

Ejemplos comunes:

```
docker compose exec backend php artisan migrate
```

```
docker compose exec backend php artisan cache:clear
```

```
docker compose exec backend php artisan db:seed
```


17. PRECAUCIONES CRÍTICAS

⚠ ADVERTENCIA: Las siguientes acciones pueden resultar en **PÉRDIDA PERMANENTE** de datos. Lea cuidadosamente antes de ejecutar.

17.1 Comando Peligroso: docker compose down -v

NUNCA ejecute este comando sin un respaldo reciente:

```
docker compose down -v
```

Este comando elimina PERMANENTEMENTE todos los volúmenes, incluyendo db_data, lo que resulta en la pérdida completa e irrecuperable de todos los datos de la base de datos.

17.2 Antes de Eliminar Volúmenes

Si necesita eliminar volúmenes, **SIEMPRE** siga estos pasos primero:

8. Crear un respaldo completo de la base de datos:

```
docker compose exec db mysqldump -u<usuario> -p<password>  
--all-databases > backup_completo_$(date +%Y%m%d_%H%M%S).sql
```

9. Verificar que el archivo de respaldo se creó correctamente y tiene contenido

10. Guardar una copia del respaldo en una ubicación segura fuera del servidor

11. Solo entonces proceder con la eliminación de volúmenes

17.3 Verificación de Espacio en Disco

Antes de iniciar los contenedores, verifique que haya suficiente espacio en disco:

```
df -h
```

Mínimo recomendado: 10 GB de espacio libre para operación normal, 20 GB para desarrollo con múltiples rebuilds.

17.4 Respaldos Programados

RECOMENDACIÓN CRÍTICA: Configure respaldos automáticos diarios de la base de datos. Ejemplo de cron job:

```
0 2 * * * cd /ruta/al/proyecto && docker compose exec -T db mysqldump  
-u<usuario> -p<password> --all-databases > /ruta/respaldos/backup_$(date  
+\\%Y\\%m\\%d).sql
```

17.5 Monitoreo de Contenedores

Verifique regularmente el estado de salud de los contenedores:

```
docker compose ps
```

```
docker stats
```

17.6 Actualización de Credenciales

IMPORTANTE: Cambie las credenciales predeterminadas de la base de datos antes de pasar a producción. Actualice el archivo `.env` con:

- Contraseñas seguras (mínimo 16 caracteres, combinación de letras, números y símbolos)
- Nombres de usuario únicos (no usar 'root' o 'admin')
- Nombres de base de datos específicos del proyecto

17.7 Lista de Verificación de Seguridad

Antes de poner en producción:

- ☐ Credenciales de base de datos actualizadas
- ☐ Sistema de respaldos automáticos configurado
- ☐ Firewall configurado correctamente
- ☐ Variables de entorno verificadas
- ☐ Logs de contenedores monitoreados
- ☐ Espacio en disco suficiente (>20 GB libre)
- ☐ Versión de respaldo de emergencia disponible

Fin del Manual

Manual de Ejecución en VM Linux con Docker

Autor: Oscar A Perez Perez

Fecha: 11 de febrero de 2026